

Class13

Stephanie Solis (A18572681)

Background

Today we will perform an RNASeq analysis of the effects of a common steroid on airway cells.

In particular, dexamethasone (hereafter just called “dex”) on different airway smooth muscle cell lines (ASM cells).

Data Import

We need two different inputs:

- countData: with genes in rows and experiments in columns
- colData: meta data that describes the columns in countData

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG00000000003	723	486	904	445	1170
ENSG00000000005	0	0	0	0	0
ENSG00000000419	467	523	616	371	582
ENSG00000000457	347	258	364	237	318
ENSG00000000460	96	81	73	66	118
ENSG00000000938	0	0	1	0	2

	SRR1039517	SRR1039520	SRR1039521
ENSG00000000003	1097	806	604
ENSG00000000005	0	0	0
ENSG00000000419	781	417	509
ENSG00000000457	447	330	324
ENSG00000000460	94	102	74
ENSG00000000938	0	0	0

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

Q1. How many genes are in this dataset?

```
nrow(counts)
```

```
[1] 38694
```

Q2. How many 'control' cell lines do we have?

```
table(metadata$dex)
```

control	treated
4	4

Differential gene expression

We have 4 replicate drug treated and control (no drug) columns/experiments in our 'counts' object. We want one "mean" value for each gene (rows) in "treated" (drug) and one mean value for each gene in "control" cols. Step 1. find all "control" columns Step 2. Extract these columns to a new object called 'control.counts' Step 3. Then calculate the mean value for each gene

```
control.inds <- metadata$dex=="control"
```

Step 2.

```
control.counts <- counts[ ,control.inds]
```

Step 3.

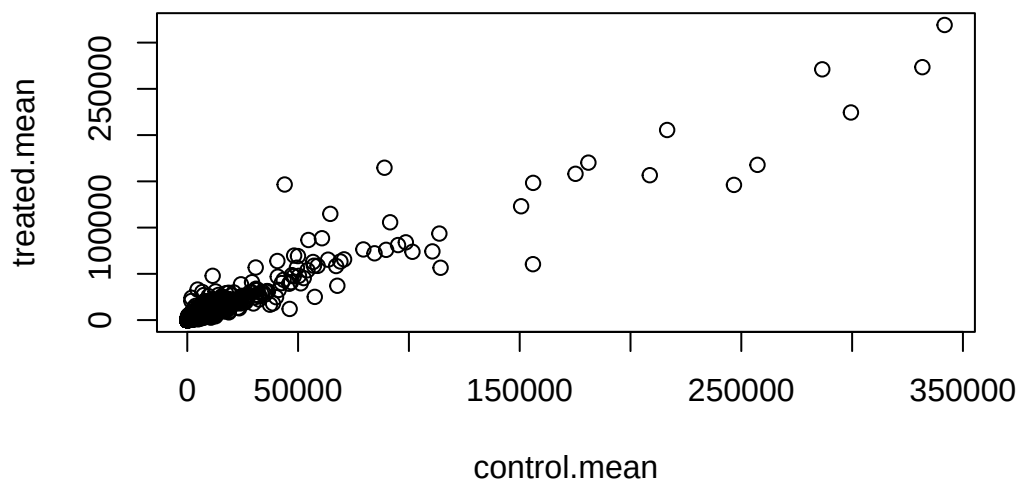
```
control.mean <- rowMeans(control.counts)
```

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

Now do the same thing for the “treated” columns/experiments

```
treated.mean <- rowMeans(counts[ ,metadata$dex=="treated"])
meancounts<- data.frame(control.mean, treated.mean)
plot(meancounts)
```

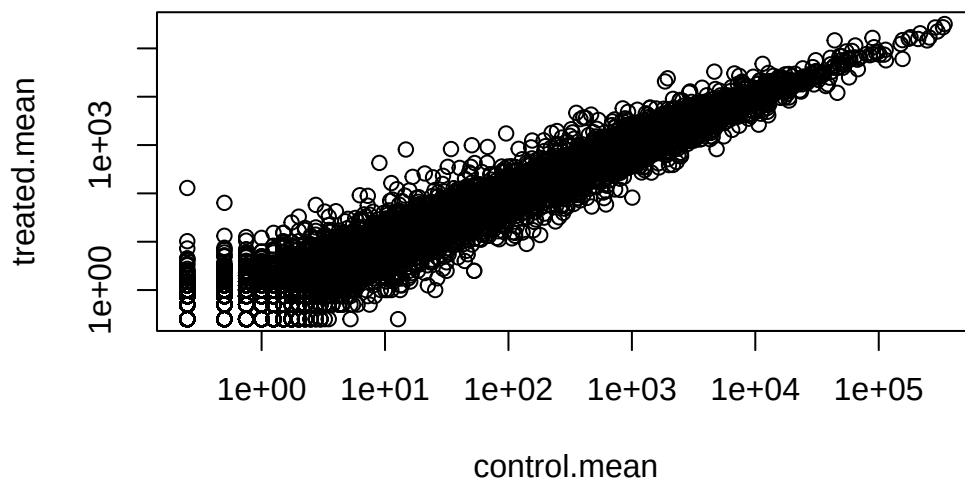


Let's log transform this count data:

```
plot(meancounts, log="xy")
```

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15032 x values ≤ 0 omitted from logarithmic plot

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15281 y values ≤ 0 omitted from logarithmic plot



N.B. We most often use $\log_2$ for this type of data as it makes the interpretation much more straightforward. Treated/Control is often called “fold-change”. if there was no change we would have a $\log_2$ -fc of zero:

```
log2(10/10)
```

```
[1] 0
```

if we had double the amount of transcript around we would have a $\log_2$ -fc of 1:

```
log2(20/10)
```

```
[1] 1
```

if we had half as much transcript around we would have a log2-fc of -1

```
log2(5/10)
```

```
[1] -1
```

Q. Calculate a log2 fold change value for all our genes and add it as a new column to our 'meancounts' object.

```
meancounts$log2f<- log2 (meancounts$treated.mean /  
meancounts$control.mean)  
head(meancounts)
```

	control.mean	treated.mean	log2f
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000938	0.75	0.00	-Inf

There are some “funky” values that come about when ever we have 0 mean count values. Typically remove these genes from any further analysis- as we can't say anything about them if we have no data for them.

```
log2(40/10)
```

```
[1] 2
```

DESeq analysis

let's do this analysis with an estimate of statistical significance using the DESeq2 package.

```
library(DESeq2)
```

```
Loading required package: S4Vectors
```

```
Loading required package: stats4
```

```
Loading required package: BiocGenerics
```

```
Loading required package: generics
```

```
Attaching package: 'generics'
```

```
The following objects are masked from 'package:base':
```

```
as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,  
setequal, union
```

```
Attaching package: 'BiocGenerics'
```

```
The following objects are masked from 'package:stats':
```

```
IQR, mad, sd, var, xtabs
```

```
The following objects are masked from 'package:base':
```

```
anyDuplicated, aperm, append, as.data.frame, basename, cbind,  
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,  
get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,  
order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,  
rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,  
unsplit, which.max, which.min
```

```
Attaching package: 'S4Vectors'
```

The following object is masked from 'package:utils':

findMatches

The following objects are masked from 'package:base':

expand.grid, I, unname

Loading required package: IRanges

Loading required package: GenomicRanges

Loading required package: Seqinfo

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats

Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,
colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
colWeightedMeans, colWeightedMedians, colWeightedSds,
colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
rowWeightedSds, rowWeightedVars

Loading required package: Biobase

Welcome to Bioconductor

Vignettes contain introductory material; view with
'browseVignettes()'. To cite Bioconductor, see
'citation("Biobase")', and for packages 'citation("pkgname")'.

Attaching package: 'Biobase'

The following object is masked from 'package:MatrixGenerics':

rowMedians

The following objects are masked from 'package:matrixStats':

anyMissing, rowMedians

DESeq (like bioconductor packages) want it's input data in a very specific way.

```
dds <- DESeqDataSetFromMatrix(countData= counts,  
colData = metadata,  
design = ~dex)
```

converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors

Run the DESeq analysis pipeline

The main function 'DESeq()'

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
res<- results(dds)
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029
	padj				
	<numeric>				
ENSG000000000003	0.163017				
ENSG000000000005	NA				
ENSG000000000419	0.175937				
ENSG000000000457	0.961682				
ENSG000000000460	0.815805				
ENSG000000000938	NA				

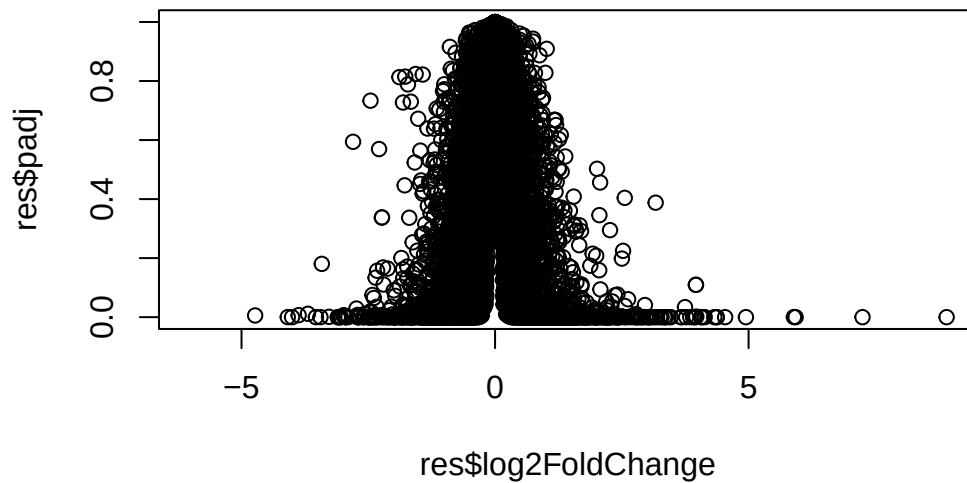
36000*0.05

[1] 1800

Volcano plot

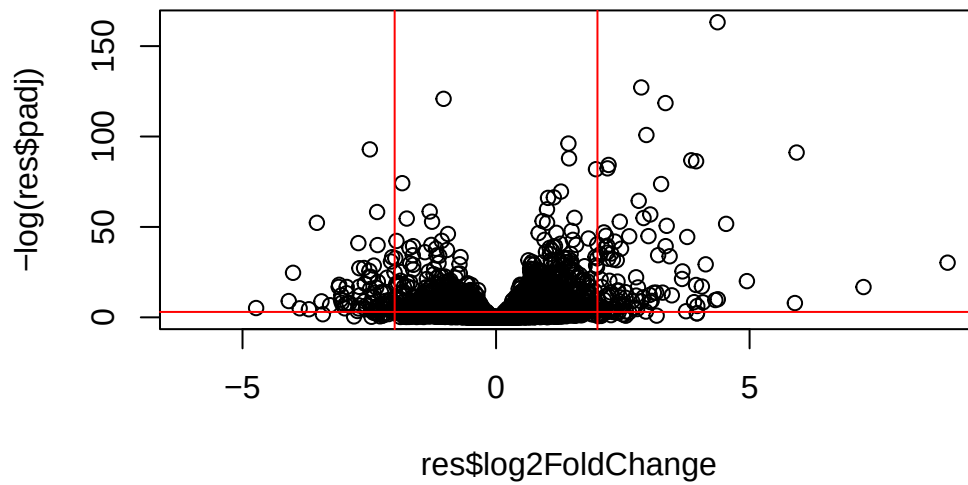
This is a main summary results figure from these kinds of studies. it is a plot of log2-foldchnage vs of (Adjusted) P-value.

```
plot(res$log2FoldChange,  
res$padj)
```



Again this y-axis is a highly needs log transforming and we can flip the y-axis with a minus sign so it looks like every other volcano plot.

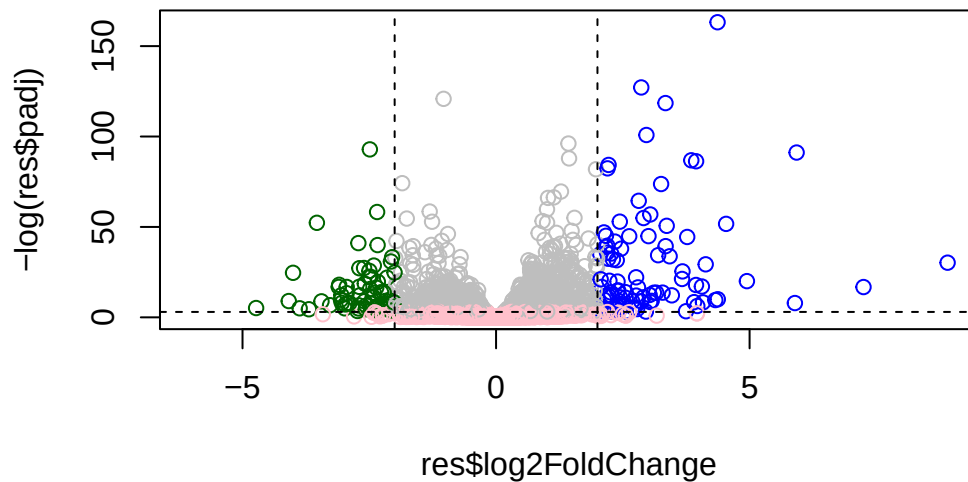
```
plot(res$log2FoldChange,  
-log(res$padj))  
abline(v=-2, col="red")  
abline(v=+2, col="red")  
abline(h=-log(0.05), col="red")
```



Adding some colors annotation

Start with default base color “gray”

```
# Start with a default base color "gray"
mycols <- rep("gray", nrow(res))
mycols[ res$log2FoldChange > 2 ] <- "blue"
mycols[ res$log2FoldChange < -2 ] <- "darkgreen"
mycols[res$padj >=0.05]<-"pink"
plot(res$log2FoldChange,
     -log(res$padj),
     col = mycols)
#Volcano
plot(res$log2FoldChange,
     -log(res$padj),
     col=mycols)
abline(v=c(-2,+2), lty=2)
abline(h=-log(0.05), lty=2)
```



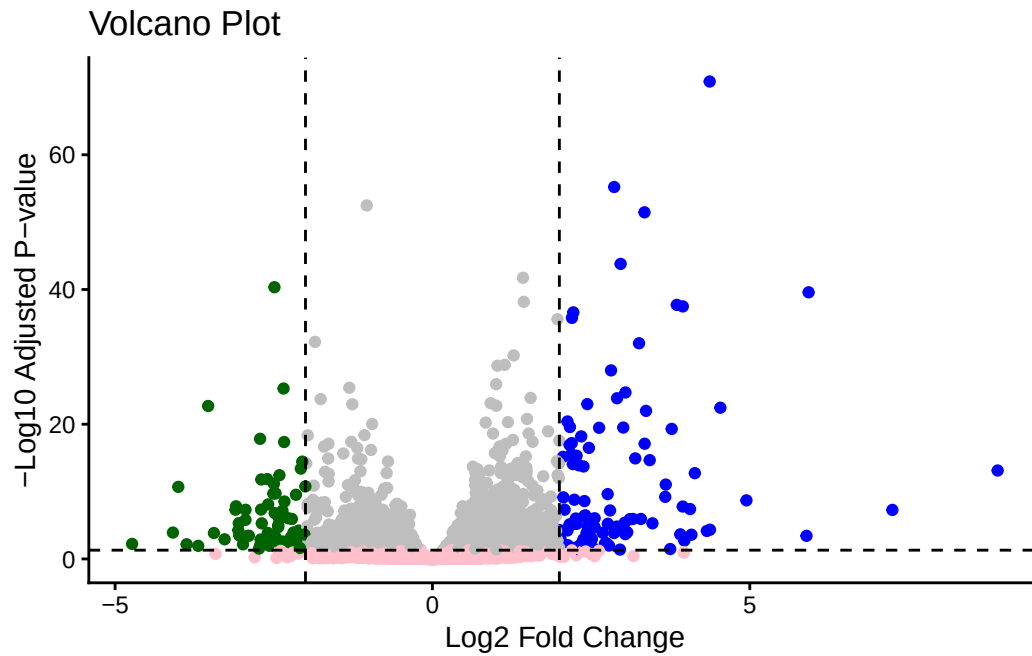
Q. Make a presentation quality ggplot version of this plot. Include clear axis labels, a clean theme, your custom colors, cut-off lines and a plot title.

```
library(ggplot2)
```

```
df <- as.data.frame(res)
df <- df[!is.na(df$log2FoldChange) & !is.na(df$padj), ]
# same color logic as your base plot
df$color <- "gray"
df$color[df$log2FoldChange > 2] <- "blue"
df$color[df$log2FoldChange < -2] <- "darkgreen"
df$color[df$padj >= 0.05] <- "pink"

ggplot(df, aes(x = log2FoldChange, y = -log10(padj))) +
  geom_point(aes(color = color), size = 1.5) +
  geom_vline(xintercept = c(-2, 2), linetype = "dashed") +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed") +
  scale_color_identity() +
  labs(
    title = "Volcano Plot",
    x = "Log2 Fold Change",
    y = "-Log10 Adjusted P-value"
  ) +
```

```
theme_classic()
```



Save our results

Write a CVS file

```
write.csv(res, file="results.csv")
```